

Data Structures (in C++)

- Hash Tables, Maps, and Dictionaries -



부산대학교 정보·의생명공학대학
정보컴퓨터공학부



Maps

Maps

- **Map**
 - A data type for storing key-value pairs
 - A key-value pair (k, v) is called an entry where k is the key and v is its corresponding value
 - Each key must be unique
 - Maps are sometimes referred to as *associative stores* or *associative containers*

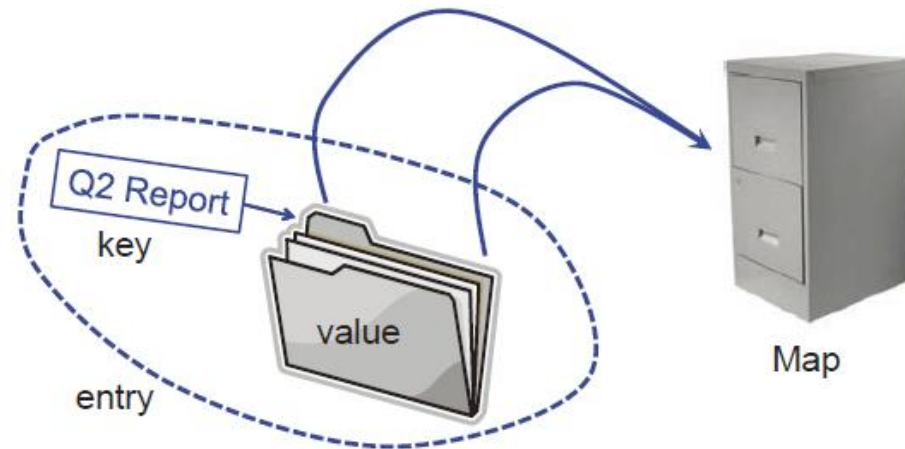


Figure 9.1: A conceptual illustration of the map ADT. Keys (labels) are assigned to values (folders) by a user. The resulting entries (labeled folders) are inserted into the map (file cabinet). The keys can be used later to retrieve or remove values.

Maps

▪ Entry Example

```
template <typename K, typename V>
class Entry {
public:
    Entry(const K& k = K(), const V& v = V())
        : _key(k), _value(v) { }
    const K& key() const { return _key; }
    const V& value() const { return _value; }
    void setKey(const K& k) { _key = k; }
    void setValue(const V& v) { _value = v; }
private:
    K _key;
    V _value;
};
```

// a (key, value) pair
// public functions
// constructor

// get key
// get value
// set key
// set value
// private data
// key
// value

Map ADT

- The map ADT M supports the following operations:

size(): Return the number of entries in M .

empty(): Return true if M is empty and false otherwise.

find(k): If M contains an entry $e = (k, v)$, with key equal to k , then return an iterator p referring to this entry, and otherwise return the special iterator end.

put(k, v): If M does not have an entry with key equal to k , then add entry (k, v) to M , and otherwise, replace the value field of this entry with v ; return an iterator to the inserted/modified entry.

erase(k): Remove from M the entry with key equal to k ; an error condition occurs if M has no such entry.

erase(p): Remove from M the entry referenced by iterator p ; an error condition occurs if p points to the end sentinel.

begin(): Return an iterator to the first entry of M .

end(): Return an iterator to a position just beyond the end of M .

Operation	Output	Map
empty()	true	$\emptyset$
put(5, A)	$p_1 : [(5, A)]$	$\{(5, A)\}$
put(7, B)	$p_2 : [(7, B)]$	$\{(5, A), (7, B)\}$
put(2, C)	$p_3 : [(2, C)]$	$\{(5, A), (7, B), (2, C)\}$
put(2, E)	$p_3 : [(2, E)]$	$\{(5, A), (7, B), (2, E)\}$
find(7)	$p_2 : [(7, B)]$	$\{(5, A), (7, B), (2, E)\}$
find(4)	end	$\{(5, A), (7, B), (2, E)\}$
find(2)	$p_3 : [(2, E)]$	$\{(5, A), (7, B), (2, E)\}$
size()	3	$\{(5, A), (7, B), (2, E)\}$
erase(5)	—	$\{(7, B), (2, E)\}$
erase(p_3)	—	$\{(7, B)\}$
find(2)	end	$\{(7, B)\}$

Example of a series of map operations

removal methods based on
a key or an iterator

Map ADT

- **C++ Implementation**

- Detailed implementation will be discussed later

```
template <typename K, typename V>
class Map {                                // map interface
public:
    class Entry;                            // a (key,value) pair
    class Iterator;                        // an iterator (and position)

    int size() const;                      // number of entries in the map
    bool empty() const;                   // is the map empty?
    Iterator find(const K& k) const;       // find entry with key k
    Iterator put(const K& k, const V& v);  // insert/replace pair (k,v)
    void erase(const K& k)                 // remove entry with key k
        throw(NonexistentElement);
    void erase(const Iterator& p);          // erase entry at p
    Iterator begin();                      // iterator to first entry
    Iterator end();                        // iterator to end entry
};
```

The STL Map

■ The STL Map Reference Manual

std::map

```
Defined in header <map>
template<
    class Key,
    class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T> >
> class map;
(1)

namespace pmr {
    template <class Key, class T, class Compare = std::less<Key>>
        using map = std::map<Key, T, Compare,
            std::pmr::polymorphic_allocator<std::pair<const Key, T>>>;
    (2) (since C++17)
}
```

std::map is a sorted associative container that contains key-value pairs with unique keys. Keys are sorted by using the comparison function Compare. Search, removal, and insertion operations have logarithmic complexity. Maps are usually implemented as red-black trees.

Everywhere the standard library uses the Compare requirements, uniqueness is determined by using the equivalence relation. In imprecise terms, two objects a and b are considered equivalent (not unique) if neither compares less than the other: !comp(a, b) && !comp(b, a).

std::map meets the requirements of *Container*, *AllocatorAwareContainer*, *AssociativeContainer* and *ReversibleContainer*.

Member types

Member type	Definition
key_type	Key
mapped_type	T
value_type	std::pair<const Key, T>
size_type	Unsigned integer type (usually std::size_t)
difference_type	Signed integer type (usually std::ptrdiff_t)
key_compare	Compare
allocator_type	Allocator
reference	value_type&
const_reference	const value_type&
pointer	Allocator::pointer (until C++11) std::allocator_traits<Allocator>::pointer (since C++11)
const_pointer	Allocator::const_pointer (until C++11) std::allocator_traits<Allocator>::const_pointer (since C++11)
iterator	LegacyBidirectionalIterator to value_type
const_iterator	LegacyBidirectionalIterator to const value_type
reverse_iterator	std::reverse_iterator<iterator>
const_reverse_iterator	std::reverse_iterator<const_iterator>

std::pair

```
Defined in header <utility>
template<
    class T1,
    class T2
> struct pair;
```

std::pair is a class template that provides a way to store two heterogeneous objects as a single unit. A pair is a specific case of a std::tuple with two elements.

If neither T1 nor T2 is a possibly cv-qualified class type with non-trivial destructor, or array thereof, the destructor of pair is trivial.

Template parameters

T1, T2 - the types of the elements that the pair stores.

Member types

Member type	Definition
first_type	T1
second_type	T2

Member objects

Member name	Type
first	T1
second	T2

key: p->first
value: p->second

The STL Map

■ The STL Map Reference Manual

std::map

Defined in header <map>

```
template<
    class Key,
    class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T> >
> class map; (1)

namespace pmr {
    template <class Key, class T, class Compare = std::less<Key>>
        using map = std::map<Key, T, Compare,
            std::pmr::polymorphic_allocator<std::pair<const Key, T>>>; (2) (since C++17)
}
```

std::map is a sorted associative container that contains key-value pairs with unique keys. Keys are sorted by using the comparison function Compare. Search, removal, and insertion operations have logarithmic complexity. Maps are usually implemented as red-black trees.

Everywhere the standard library uses the Compare requirements, uniqueness is determined by using the equivalence relation. In imprecise terms, two objects a and b are considered equivalent (not unique) if neither compares less than the other: !comp(a, b) && !comp(b, a).

std::map meets the requirements of *Container*, *AllocatorAwareContainer*, *AssociativeContainer* and *ReversibleContainer*.

Member types

Member type	Definition
key_type	Key
mapped_type	T
value_type	std::pair<const Key, T>
size_type	Unsigned integer type (usually std::size_t)
difference_type	Signed integer type (usually std::ptrdiff_t)
key_compare	Compare
allocator_type	Allocator
reference	value_type&
const_reference	const value_type&
pointer	Allocator::pointer (until C++11) std::allocator_traits<Allocator>::pointer (since C++11)
const_pointer	Allocator::const_pointer (until C++11) std::allocator_traits<Allocator>::const_pointer (since C++11)
iterator	LegacyBidirectionalIterator to value_type
const_iterator	LegacyBidirectionalIterator to const value_type
reverse_iterator	std::reverse_iterator<iterator>
const_reverse_iterator	std::reverse_iterator<const_iterator>

Element access

at	access specified element with bounds checking (public member function)
operator[]	access or insert specified element (public member function)

Iterators

begin cbegin (C++11)	returns an iterator to the beginning (public member function)
end cend (C++11)	returns an iterator to the end (public member function)
rbegin crbegin (C++11)	returns a reverse iterator to the beginning (public member function)
rend crend (C++11)	returns a reverse iterator to the end (public member function)

Capacity

empty	checks whether the container is empty (public member function)
size	returns the number of elements (public member function)
max_size	returns the maximum possible number of elements (public member function)

Modifiers

clear	clears the contents (public member function)
insert	inserts elements or nodes (since C++17) (public member function)
insert_or_assign (C++17)	inserts an element or assigns to the current element if the key already exists (public member function)
emplace (C++11)	constructs element in-place (public member function)
emplace_hint (C++11)	constructs elements in-place using a hint (public member function)
try_emplace (C++17)	inserts in-place if the key does not exist, does nothing if the key exists (public member function)
erase	erases elements (public member function)
swap	swaps the contents (public member function)

<https://en.cppreference.com/w/cpp/container/map>

The STL Map

▪ The STL Map Example

`size()`: Return the number of elements in the map.

`empty()`: Return true if the map is empty and false otherwise.

`find(k)`: Find the entry with key *k* and return an iterator to it; if no such key exists return end.

`operator[k]`: Produce a reference to the value of key *k*; if no such key exists, create a new entry for key *k*.

`insert(pair(k, v))`: Insert pair (*k*, *v*), returning an iterator to its position.

`erase(k)`: Remove the element with key *k*.

`erase(p)`: Remove the element referenced by iterator *p*.

`begin()`: Return an iterator to the beginning of the map.

`end()`: Return an iterator just past the end of the map.

```
map<string, int> myMap; // a (string,int) map
map<string, int>::iterator p; // an iterator to the map
myMap.insert(pair<string, int>("Rob", 28)); // insert ("Rob",28)
myMap["Joe"] = 38; // insert("Joe",38)
myMap["Joe"] = 50; // change to ("Joe",50)
myMap["Sue"] = 75; // insert("Sue",75)
p = myMap.find("Joe"); // *p = ("Joe",50)
myMap.erase(p); // remove ("Joe",50)
myMap.erase("Sue"); // remove ("Sue",75)
p = myMap.find("Joe");
if (p == myMap.end()) cout << "nonexistent\n"; // outputs: "nonexistent"
for (p = myMap.begin(); p != myMap.end(); ++p) { // print all entries
    cout << "(" << p->first << ", " << p->second << ")\n";
}
```

List-based Map

- **Implementation with a Doubly Linked List**
 - Fundamental operations take $O(n)$ due to searching

Algorithm find(k):

Input: A key k

Output: The position of the matching entry of L , or end if there is no key k in L

```
for each position  $p \in [L.begin(), L.end())$  do
    if  $p.key() = k$  then
        return  $p$ 
return end          {there is no entry with key equal to  $k$ }
```

Algorithm put(k, v):

Input: A key-value pair (k, v)

Output: The position of the inserted/modified entry

```
for each position  $p \in [L.begin(), L.end())$  do
    if  $p.key() = k$  then
         $*p \leftarrow (k, v)$ 
        return  $p$           {return the position of the modified entry}
 $p \leftarrow L.insertBack((k, v))$ 
 $n \leftarrow n + 1$         {increment variable storing number of entries}
return  $p$                  {return the position of the inserted entry}
```

Algorithm erase(k):

Input: A key k

Output: None

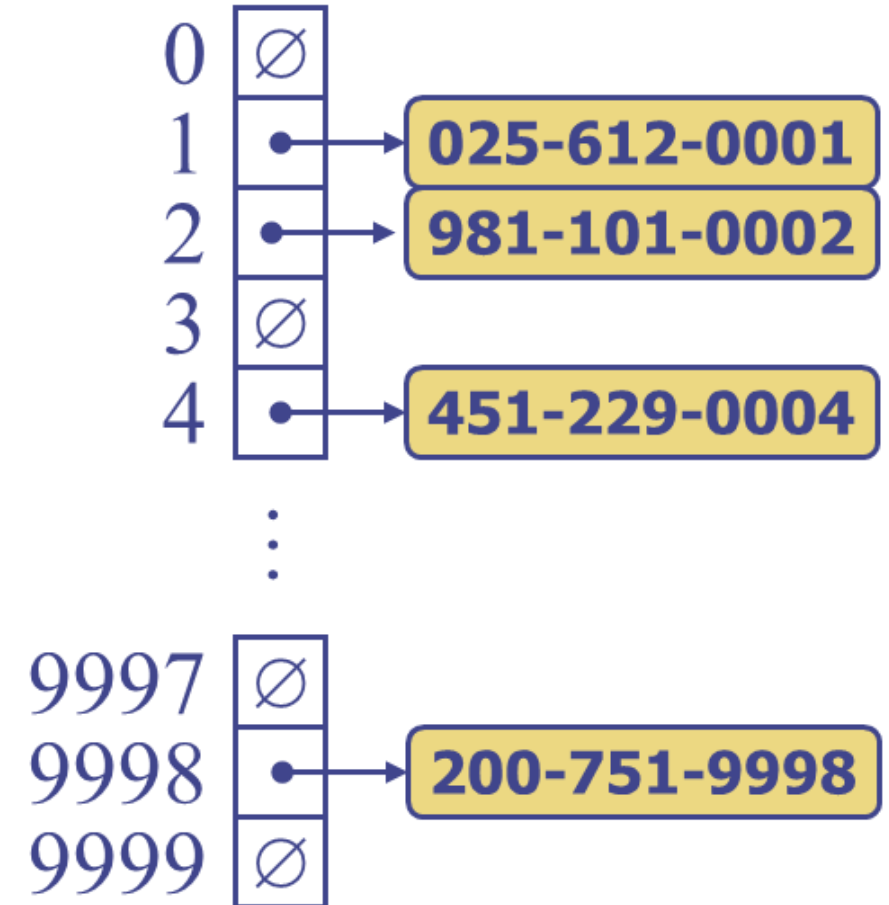
```
for each position  $p \in [L.begin(), L.end())$  do
    if  $p.key() = k$  then
         $L.erase(p)$ 
         $n \leftarrow n - 1$     {decrement variable storing number of entries}
```

Hash Tables

Hash Tables

▪ Hash Table

- A data type which stores data in an associative manner
- The running time of map operations is $O(n)$ in the worst case
- A hash table can usually perform these operations in $O(1)$ expected time
- A hash table consists of two major components:
 - a *bucket array*
 - a *hash function*



Hash Tables

▪ Bucket Arrays

- An array A of size N where each cell is thought of as a *bucket* (i.e., a collection of (k, v) pairs)
- N defines the capacity of the array A
- Each bucket holds at most one entry
 - Searches, insertions, and removals take $O(1)$ time
- Keys are required to be integers in the range $[0, N-1]$, which is often not the case

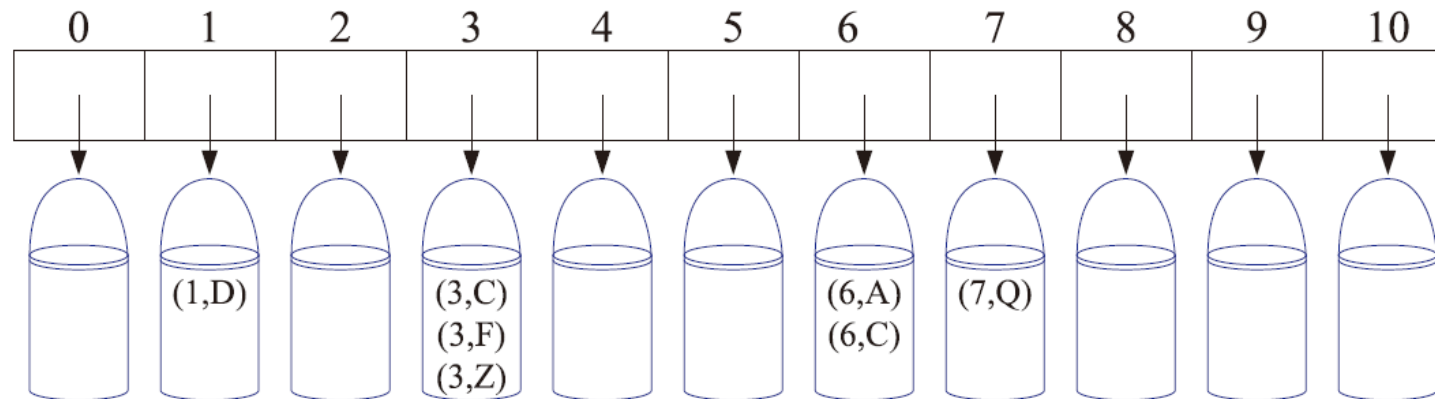
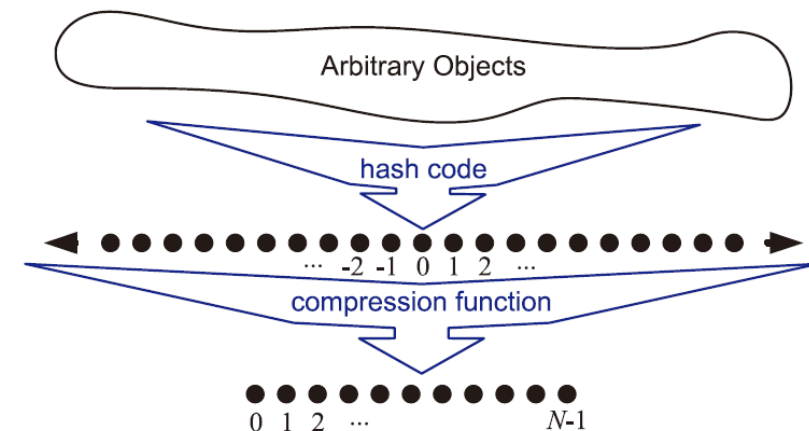


Figure 9.2: A bucket array of size 11 for the entries (1,D), (3,C), (3,F), (3,Z), (6,A), (6,C), and (7,Q).

Hash Tables

▪ Hash Functions

- A *hash function* $h(\cdot)$ maps each key k in a map to an integer in the range $[0, N-1]$
- Use the hash function value $h(k)$ as an index into the bucket array (instead of the key k)
 $A[h(k)] = (k, v)$
- A collision occurs if there are two or more keys with the same hash value
- A hash function is “good” if it minimizes collisions as much as possible
- A hash function consists of two actions:
 - Mapping the key k to an integer called the *hash code*
 - Mapping the hash code to an integer within the range $[0, N-1]$ using a *compression function*



Hash Codes

■ Hash Codes

- The integer assigned to a key k is called the hash code for k
- Need not be in the range $[0, N-1]$
- Need to avoid collisions as much as possible
- $h(k_1) = h(k_2)$ if $k_1 = k_2$ What about the opposite?
- The following discussions assume that the number of bits of each type is known

`std::numeric_limits<T>::digits`

```
static const int digits;           (until C++11)
static constexpr int digits;      (since C++11)
```

The value of `std::numeric_limits<T>::digits` is the number of digits in base-radix that can be represented by the type `T` without change. For integer types, this is the number of bits not counting the sign bit and the padding bits (if any). For floating-point types, this is the digits of the mantissa (for IEC 559/IEEE 754 implementations, this is the number of digits stored for the mantissa plus one, because the mantissa has an implicit leading 1 and binary point).

Standard specializations

T	value of <code>std::numeric_limits<T>::digits</code> (assuming no padding bits)
/* non-specialized */	0
<code>bool</code>	1
<code>char</code>	<code>CHAR_BIT - std::numeric_limits<char>::is_signed</code>
<code>signed char</code>	<code>CHAR_BIT - 1</code>
<code>unsigned char</code>	<code>CHAR_BIT</code>
<code>wchar_t</code>	<code>CHAR_BIT*sizeof(wchar_t) - std::numeric_limits<wchar_t>::is_signed</code>
<code>char8_t</code>	<code>CHAR_BIT</code>
<code>char16_t</code>	<code>CHAR_BIT*sizeof(char16_t)</code>
<code>char32_t</code>	<code>CHAR_BIT*sizeof(char32_t)</code>
<code>short</code>	<code>CHAR_BIT*sizeof(short)-1</code>
<code>unsigned short</code>	<code>CHAR_BIT*sizeof(short)</code>
<code>int</code>	<code>CHAR_BIT*sizeof(int)-1</code>
<code>unsigned int</code>	<code>CHAR_BIT*sizeof(int)</code>
<code>long</code>	<code>CHAR_BIT*sizeof(long)-1</code>
<code>unsigned long</code>	<code>CHAR_BIT*sizeof(long)</code>
<code>long long</code>	<code>CHAR_BIT*sizeof(long long)-1</code>
<code>unsigned long long</code>	<code>CHAR_BIT*sizeof(long long)</code>
<code>float</code>	<code>FLT_MANT_DIG</code>
<code>double</code>	<code>DBL_MANT_DIG</code>
<code>long double</code>	<code>LDBL_MANT_DIG</code>

https://en.cppreference.com/w/cpp/types/numeric_limits/digits

Hash Codes

- **Integer Cast**

- Reinterpret the bits of the key as an integer
- Suitable for keys of length less than or equal to the number of bits of the integer (*e.g.*, byte, short, int, and float in C++)

- **Component Sum**

- Partition the bits of the key into components of fixed length and sum the components
- Suitable for keys of greater than or equal to the number of bits of the integer (*e.g.*, long and double in C++)
- Can be extended to any object whose binary representation can be views as a tuple of integers

$$k = (x_0, x_1, \dots, x_{m-1}) \rightarrow h(k) = \sum_{i=0}^{m-1} x_i$$

Hash Codes

Integer Cast

Key	Bits	Hash Code
(byte) 22	(8-bit) 00010110	22
(short) 14	(16-bit) 00000000 00001110	14
(int) 7	(32-bit) 00000000 00000000 00000000 00000111	7
(float) 2.2	(32-bit) 01000000 00001100 11001100 11001101	1,074,580,685

Component Sum

https://www.binaryconvert.com/convert_double.html

Key	Bits	Hash Code
(long) -2	(64-bit) 11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110 11111111 11111111 11111111 11111101	4,294,967,293
(double) 3.7	(64-bit) 01000000 00001101 10011001 10011001 10011001 10011001 10011001 10011010 11011001 10100111 00110011 00110011	3,651,613,491 (1,074,633,113+2,576,980,378)

Hash Codes

- **Polynomial Hash Codes**

- The summation hash code is not suitable when the ordering is important (*e.g., stop, tops, pots, and spot*)

- Takes into consideration the x_i 's position ($a \neq 0$)

$$x_0a^{m-1} + x_1a^{m-2} + \cdots + x_{m-2}a + x_{m-1}$$

$$= x_{m-1} + a \left(x_{m-2} + a \left(x_{m-3} + \cdots + a \left(x_2 + a \left(x_1 + ax_0 \right) \right) \right) \right)$$

- 33, 37, 39, and 41 are good choices for a

Hash Codes

■ Cyclic Shift Hash Codes

- Replaces multiplication by a with a cyclic shift of a partial sum

```
int hashCode(const char* p, int len) {           // hash a character array
    unsigned int h = 0;
    for (int i = 0; i < len; i++) {
        h = (h << 5) | (h >> 27);               // 5-bit cyclic shift
        h += (unsigned int) p[i];               // add in next character
    }
    return hashCode(int(h));                     ← Assume that we have a hashCode function for an integer
}
```

- 8-bit toy example

14 = 00001110

14 << 3 =

14 >> 5 =

14 << 3 | 14 >> 5 =

Simple addition →

Shift	Collisions		Shift	Collisions	
	Total	Max		Total	Max
0	23739	86	9	18	2
1	10517	21	10	277	3
2	2254	6	11	453	4
3	448	3	12	43	2
4	89	2	13	13	2
5	4	2	14	135	3
6	6	2	15	1082	6
7	14	2	16	8760	9
8	105	2			

Table 9.1: Comparison of collision behavior for the cyclic shift variant of the polynomial hash code as applied to a list of just over 25,000 English words. The “Total” column records the total number of collisions and the “Max” column records the maximum number of collisions for any one hash code. Note that, with a cyclic shift of 0, this hash code reverts to the one that simply sums all the characters.

Hash Codes

▪ Hashing Floating-Point Quantities

- Direct integer conversion will truncate fractional part of the floating-point value

```
int hashCode(const float& x) {           // hash a float
    int len = sizeof(x);
    const char* p = reinterpret_cast<const char*>(&x);
    return hashCode(p, len);
}
```

2.50 = 01000000 00100000 00000000 00000000

2.25 = 01000000 00010000 00000000 00000000

Compression Functions

- The hash code for a key k is typically not suitable for immediate use with a bucket array
- The compression step maps the hash code into the range $[0, N-1]$

- **The Division Method**

- The compression function of the division method is:

$$h(k) = |k| \bmod N$$

- A prime number N helps spread out the distribution of hashed values

Hash codes: {200, 205, 210, 215, 220, ..., 600}

$N = 100$

$N = 101$

- A good hash function ensures the probability of collision to be $1/N$

Compression Functions

- **The MAD Method**

- Multiply, Add, and Divide (MAD) compression function is:

$$h(k) = |ak + b| \bmod N$$

- N is a prime number, a and b are nonnegative integers ($a \bmod N \neq 0$)

- **Collision-Handling**

- A good hash function still has the collision probability $1/N$
 - Cannot simply insert a new entry if collision occurs

Collision-Handling Schemes

▪ Separate Chaining

- Each bucket $A[i]$ stores a small map M_i (using a list) where $h(k) = i$

Algorithm find(k):

Output: The position of the matching entry of the map, or end if there is no key k in the map

return $A[h(k)].find(k)$ {delegate the find(k) to the list-based map at $A[h(k)]$ }

Algorithm put(k, v):

$p \leftarrow A[h(k)].put(k, v)$ {delegate the put to the list-based map at $A[h(k)]$ }

$n \leftarrow n + 1$

return p

Algorithm erase(k):

Output: None

$A[h(k)].erase(k)$ {delegate the erase to the list-based map at $A[h(k)]$ }

$n \leftarrow n - 1$

Algorithm find(k):

Input: A key k

Output: The position of the matching entry of L , or end if there is no key k in L

for each position $p \in [L.begin(), L.end())$ **do**

if $p.key() = k$ **then**

return p

return end {there is no entry with key equal to k }

Algorithm put(k, v):

Input: A key-value pair (k, v)

Output: The position of the inserted/modified entry

for each position $p \in [L.begin(), L.end())$ **do**

if $p.key() = k$ **then**

$*p \leftarrow (k, v)$

return p

 {return the position of the modified entry}

$p \leftarrow L.insertBack((k, v))$

$n \leftarrow n + 1$ {increment variable storing number of entries}

return p {return the position of the inserted entry}

Algorithm erase(k):

Input: A key k

Output: None

for each position $p \in [L.begin(), L.end())$ **do**

if $p.key() = k$ **then**

$L.erase(p)$

$n \leftarrow n - 1$

 {decrement variable storing number of entries}

**List-based Map
Implementation**

Collision-Handling Schemes

- **Separate Chaining**

- Requires the use of an auxiliary data structure (*i.e.*, list)

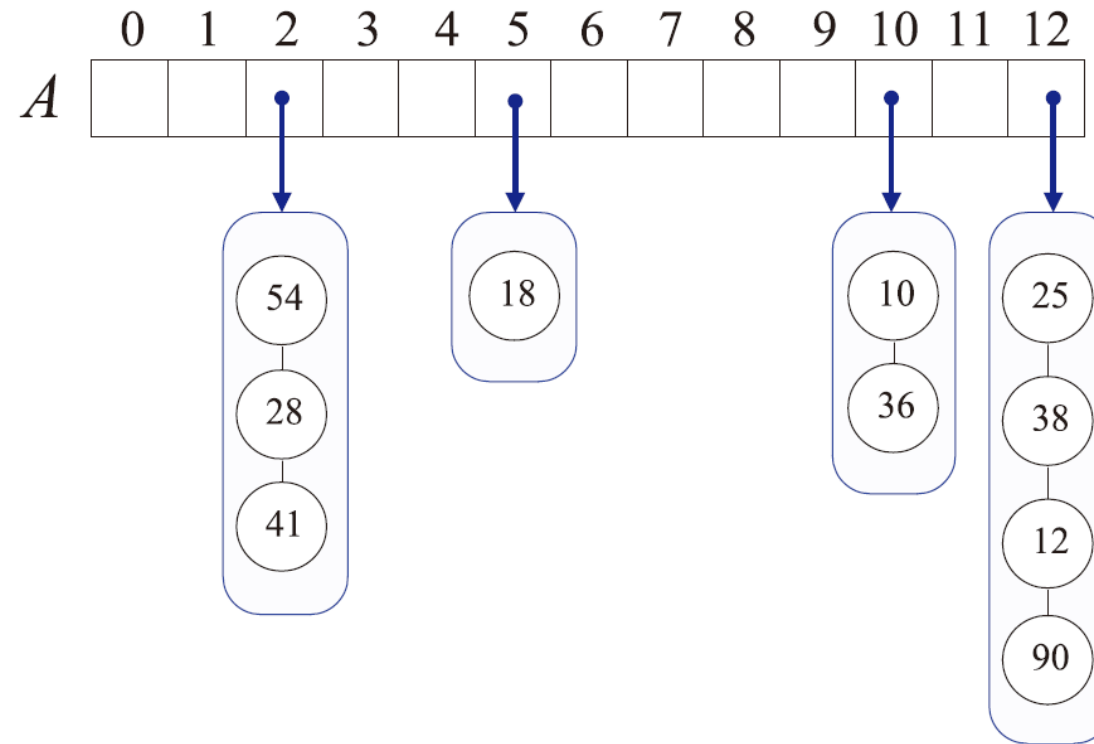


Figure 9.4: A hash table of size 13, storing 10 entries with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

Collision-Handling Schemes

- **Load Factor**

- We expect each bucket to be of size n/N with a good hash function
- The *load factor* λ of the hash table should be bounded by a small constant (below 1)
- Expected running time of *find*, *put*, and *erase* operations is $O(\lceil n/N \rceil)$
- If n is $O(N)$, these operations run in $O(1)$

Collision-Handling Schemes

▪ Open Addressing

- Stores each entry directly in a bucket, at most one entry per bucket
- Saves spaces because no auxiliary structures are used
- The load factor is always at most 1

▪ Linear Probing

- If a bucket $A[i]$ is already occupied where $i = h(k)$, try next at $A[(i + 1) \bmod N]$
- If the next one is occupied too, try next at $A[(i + 2) \bmod N]$
- Repeat until an empty bucket is found and insert a new entry in the bucket

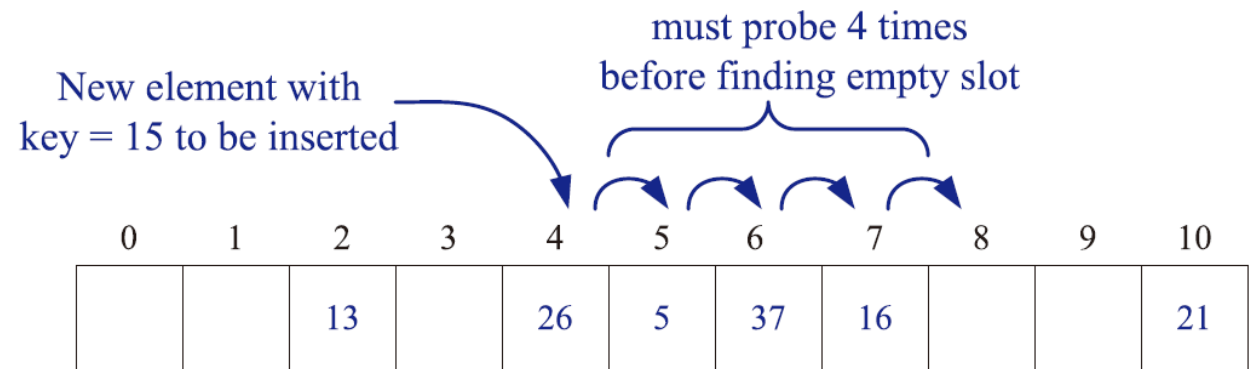


Figure 9.5: An insertion into a hash table using linear probing to resolve collisions. Here we use the compression function $h(k) = k \bmod 11$.

Collision-Handling Schemes

■ Linear Probing

- Assume a $get(k, v)$ operation returns a pair (k, v) if it exists
- Searching starts from $A[h(k)]$ and examine consecutive buckets until an empty bucket or the pair (k, v) is found
- $erase(k)$ may need shifting of entries
- Replacing a deleted entry with a special "*available*" marker resolves the issue
- Searching for a key k skips "*available*" markers and continues probing
- $put(k, v)$ should remember an available cell encountered during the search
- Clustered entry problem exists

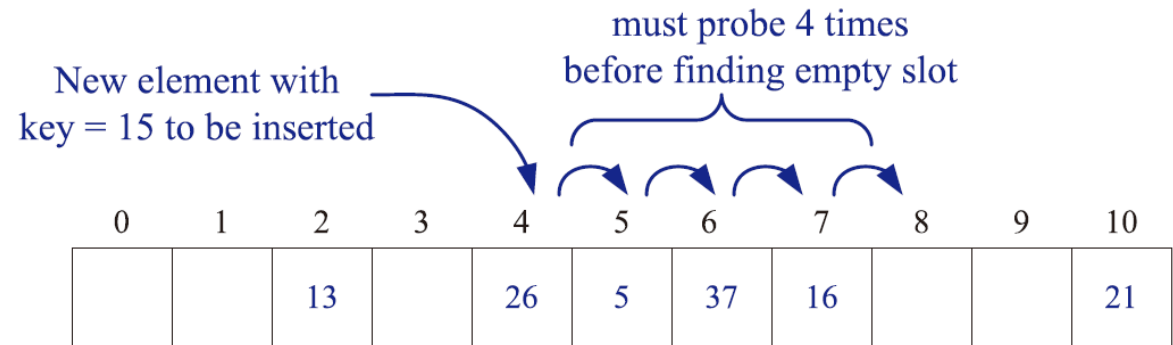


Figure 9.5: An insertion into a hash table using linear probing to resolve collisions. Here we use the compression function $h(k) = k \bmod 11$.

Collision-Handling Schemes

▪ Quadratic Probing

- If a bucket $A[i]$ is already occupied where $i = h(k)$, try next at $A[(i + f(j)) \bmod N]$ where $f(j) = j^2$
- Still not free from the clustering patterns
- If N is not a prime, the quadratic-probing may not find an empty bucket even if one exists

▪ Double Hashing

- Choose a secondary hash function $h'(\cdot)$
- Try next at $A[(i + f(j)) \bmod N]$ where $f(j) = jh'(k)$
- A common choice is $h'(k) = q - (k \bmod q)$ for some prime number $q < N$ where N is also a prime

Collision-Handling Schemes

▪ Double Hashing

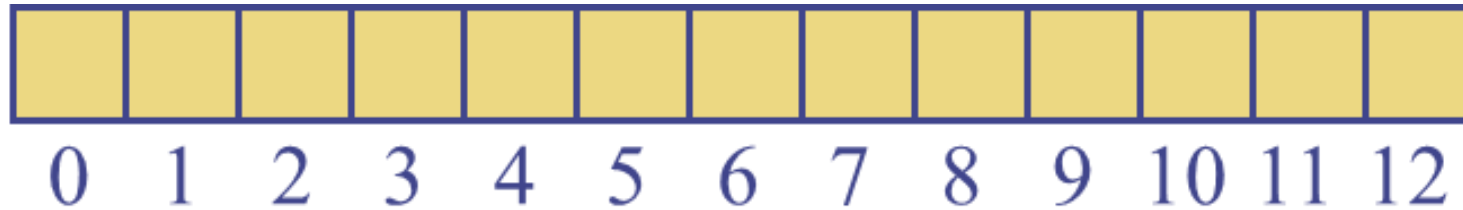
- Consider a hash table storing integer keys that handles collision with double hashing

$$N = 13$$

$$h(k) = k \bmod 13$$

$$d(k) = 7 - k \bmod 7$$

- Insert keys 18, 41, 22, 44, 59, 32, 31, and 73 in this order



k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

Load Factors and Rehashing

- Open addressing schemes save some space but not necessarily faster than the separate-chaining method
- The load factor $\lambda = n/N$ should be kept below 1
 - $\lambda < 0.5$ for the open-addressing schemes
 - $\lambda < 0.9$ for the separate chaining
- At the limit when λ is close to 1, all map operations have linear expected running times
- **Rehashing**
 - If the load factor grows above a threshold, resize the table (*i.e.*, double the size)
 - Define a new hash function (*e.g.*, new parameters for the MAD method)
 - All the objects are inserted into the new resized table

Hash Tables

■ C++ Implementation

```
template <typename K, typename V, typename H>
class HashMap {
public:
    // public types
    typedef Entry<const K,V> Entry;      // a (key,value) pair
    class Iterator;                       // a iterator/position
    // public functions
    HashMap(int capacity = 100);          // constructor
    int size() const;                     // number of entries
    bool empty() const;                   // is the map empty?
    Iterator find(const K& k);             // find entry with key k
    Iterator put(const K& k, const V& v);  // insert/replace (k,v)
    void erase(const K& k);                // remove entry with key k
    void erase(const Iterator& p);         // erase entry at p
    Iterator begin();                      // iterator to first entry
    Iterator end();                        // iterator to end entry
protected:
    // protected types
    typedef std::list<Entry> Bucket;      // a bucket of entries
    typedef std::vector<Bucket> BktArray; // a bucket array
    // ...insert HashMap utilities here
private:
    int n;                                // number of entries
    H hash;                                // the hash comparator
    BktArray B;                            // bucket array
public:
    // ...insert Iterator class declaration here
};
```

```
Iterator finder(const K& k);              // find utility
Iterator inserter(const Iterator& p, const Entry& e); // insert utility
void eraser(const Iterator& p);           // remove utility
typedef typename BktArray::iterator Bltor; // bucket iterator
typedef typename Bucket::iterator Eltor;  // entry iterator
static void nextEntry(Iterator& p)        // bucket's next entry
{ ++p.ent; }
static bool endOfBkt(const Iterator& p)    // end of bucket?
{ return p.ent == p.bkt->end(); }
```

Code Fragment 9.7: Declarations of utilities to be inserted into HashMap.

```
class Iterator {                                // an iterator (& position)
private:
    Eltor ent;                                // which entry
    Bltor bkt;                                // which bucket
    const BktArray* ba;                       // which bucket array
public:
    Iterator(const BktArray& a, const Bltor& b, const Eltor& q = Eltor())
        : ent(q), bkt(b), ba(&a) { }
    Entry& operator*() const;                  // get entry
    bool operator==(const Iterator& p) const;  // are iterators equal?
    Iterator& operator++();                    // advance to next entry
    friend class HashMap;                     // give HashMap access
};
```

Code Fragment 9.8: Declaration of the Iterator class for HashMap.

Hash Tables

■ C++ Implementation

- Qualifier and template statements are omitted

```

/* HashMap<K,V,H> :: */
Entry& Iterator::operator*() const // get entry
{ return *ent; }

/* HashMap<K,V,H> :: */
bool Iterator::operator==(const Iterator& p) const { // are iterators equal?
    if (ba != p.ba || bkt != p.bkt) return false; // ba or bkt differ?
    else if (bkt == ba->end()) return true; // both at the end?
    else return (ent == p.ent); // else use entry to decide
}

/* HashMap<K,V,H> :: */
Iterator& Iterator::operator++() { // advance to next entry
    ++ent; // next entry in bucket
    if (endOfBkt(*this)) { // at end of bucket?
        ++bkt; // go to next bucket
        while (bkt != ba->end() && bkt->empty()) // find nonempty bucket
            ++bkt;
        if (bkt == ba->end()) return *this; // end of bucket array?
        ent = bkt->begin(); // first nonempty entry
    }
    return *this; // return self
}

/* HashMap<K,V,H> :: */
Iterator end() // iterator to end
{ return Iterator(B, B.end()); }

/* HashMap<K,V,H> :: */
Iterator begin() { // iterator to front
    if (empty()) return end(); // empty - return end
    BItor bkt = B.begin(); // else search for an entry
    while (bkt->empty()) ++bkt; // find nonempty bucket
    return Iterator(B, bkt, bkt->begin()); // return first of bucket
}

/* HashMap<K,V,H> :: */
HashMap(int capacity) : n(0), B(capacity) { } // constructor

/* HashMap<K,V,H> :: */
int size() const { return n; } // number of entries

/* HashMap<K,V,H> :: */
bool empty() const { return size() == 0; } // is the map empty?
    
```

Hash Tables

■ C++ Implementation

```
/* HashMap<K,V,H> :: */  
Iterator finder(const K& k) {  
    int i = hash(k) % B.size();  
    Btor bkt = B.begin() + i;  
    Iterator p(B, bkt, bkt->begin());  
    while (!endOfBkt(p) && (*p).key() != k)  
        nextEntry(p);  
    return p;  
}
```

```
/* HashMap<K,V,H> :: */  
Iterator find(const K& k) {  
    Iterator p = finder(k);  
    if (endOfBkt(p))  
        return end();  
    else  
        return p;  
}
```

// find utility

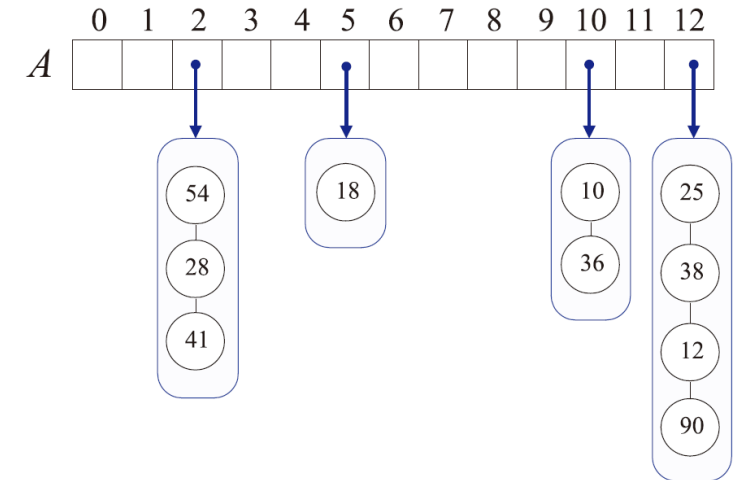
// get hash index i
// the ith bucket
// start of ith bucket
// search for k

// return final position

// find key

// look for k
// didn't find it?
// return end iterator

// return its position



Hash Tables

■ C++ Implementation

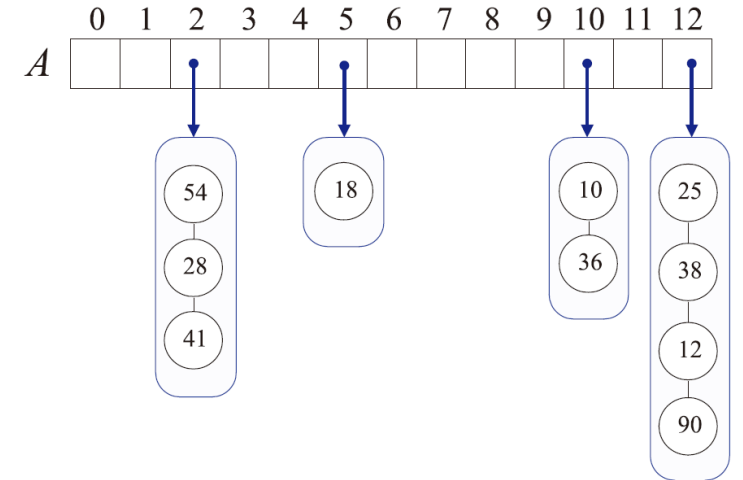
```
/* HashMap<K,V,H> :: */  
Iterator inserter(const Iterator& p, const Entry& e) {  
    Eltor ins = p.bkt->insert(p.ent, e);  
    n++;  
    return Iterator(B, p.bkt, ins);  
}  
  
/* HashMap<K,V,H> :: */  
Iterator put(const K& k, const V& v) {  
    Iterator p = finder(k);  
    if (endOfBkt(p)) {  
        return inserter(p, Entry(k, v));  
    }  
    else {  
        p.ent->setValue(v);  
        return p;  
    }  
}
```

// insert utility
// insert before p
// one more entry
// return this position

// insert/replace (v,k)

// search for k
// k not found?
// insert at end of bucket

// found it?
// replace value with v
// return this position



Hash Tables

■ C++ Implementation

```
/* HashMap<K,V,H> :: */  
void eraser(const Iterator& p) {  
    p.bkt->erase(p.ent);  
    n--;  
}
```

```
/* HashMap<K,V,H> :: */  
void erase(const Iterator& p)  
    { eraser(p); }
```

```
/* HashMap<K,V,H> :: */  
void erase(const K& k) {  
    Iterator p = finder(k);  
    if (endOfBkt(p))  
        throw NonexistentElement("Erase of nonexistent"); // ...error  
    eraser(p);  
}
```

```
// remove utility
```

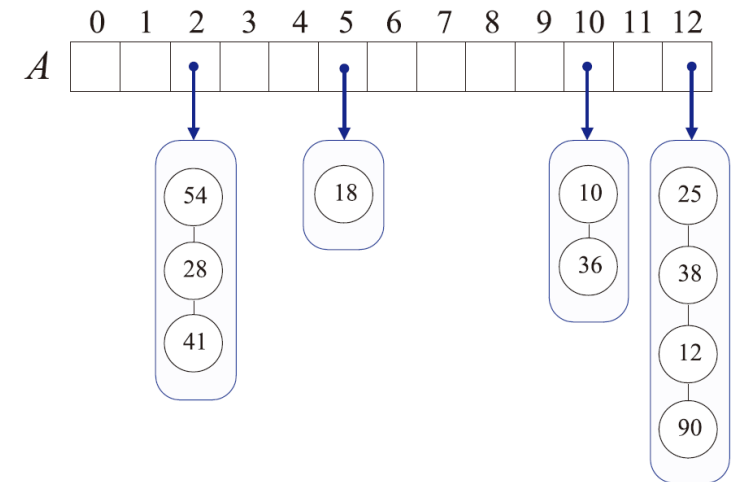
```
// remove entry from bucket  
// one fewer entry
```

```
// remove entry at p
```

```
// remove entry with key k
```

```
// find k  
// not found?
```

```
// remove it
```



Ordered Maps

Ordered Maps

- **Ordered Map**

- A data type for storing key-value pairs that are sorted according to some total order
- A comparator provides the order relation among keys

- The ordered map ADT supports the following operations:

`firstEntry( $k$ )`: Return an iterator to the entry with smallest key value; if the map is empty, it returns end.

`lastEntry( $k$ )`: Return an iterator to the entry with largest key value; if the map is empty, it returns end.

`ceilingEntry( $k$ )`: Return an iterator to the entry with the least key value greater than or equal to k ; if there is no such entry, it returns end.

`floorEntry( $k$ )`: Return an iterator to the entry with the greatest key value less than or equal to k ; if there is no such entry, it returns end.

`lowerEntry( $k$ )`: Return an iterator to the entry with the greatest key value less than k ; if there is no such entry, it returns end.

`higherEntry( $k$ )`: Return an iterator to the entry with the least key value greater than k ; if there is no such entry, it returns end.

Ordered Maps

▪ Ordered Search Tables

- Store the map's entries in a vector L in increasing order of the keys
- A vector provides better worst-case running time compared to that of a linked list
- Referred to as an *ordered search table*
- $insert(k, v)$ and $erase(k)$ take $O(n)$ due to the shift operation of all the entries with key greater than k

0	1	2	3	4	5	6	7	8	9	10
4	6	9	12	15	16	18	28	34		

Figure 9.6: Realization of a map by means of an ordered search table. We show only the keys for this map in order to highlight their ordering.

Ordered Maps

▪ Binary Search

- Accessing an element of L by its index takes $O(1)$ time
- An entry is a candidate if we cannot rule out that this entry has key equal to k
- All the candidate entries have index at least low and at most $high$ in L
- Compare k to the key of the median candidate e with index $mid = \lfloor (low + high)/2 \rfloor$
- Consider three cases:
 - If $k = e.key()$, then we have found the entry we were looking for, and the search terminates successfully returning e
 - If $k < e.key()$, then we recur on the first half of the vector, that is, on the range of indices from low to $mid - 1$
 - If $k > e.key()$, we recur on the range of indices from $mid + 1$ to $high$

Ordered Maps

■ Binary Search

Algorithm BinarySearch($L, k, low, high$):

Input: An ordered vector L storing n entries and integers low and $high$

Output: An entry of L with key equal to k and index between low and $high$, if such an entry exists, and otherwise the special sentinel end

if $low > high$ **then**

return end

else

$mid \leftarrow \lfloor (low + high) / 2 \rfloor$

$e \leftarrow L.at(mid)$

if $k = e.key()$ **then**

return e

else if $k < e.key()$ **then**

return BinarySearch($L, k, low, mid - 1$)

else

return BinarySearch($L, k, mid + 1, high$)

Code Fragment 9.18: Binary search in an ordered vector.

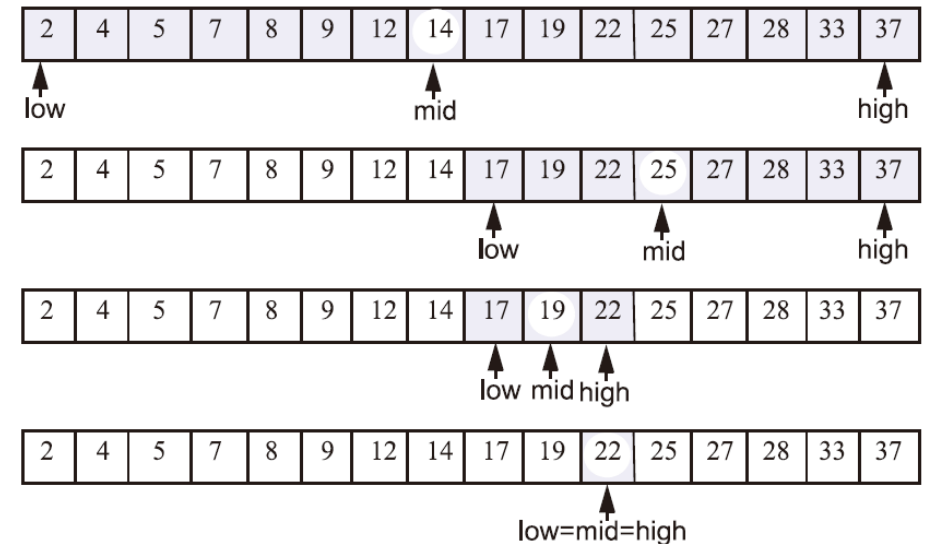


Figure 9.7: Example of a binary search to perform operation find(22), in a map with integer keys, implemented with an ordered vector. For simplicity, we show the keys, not the whole entries.

Ordered Maps

■ Binary Search

- The running time is proportional to the number of recursive calls performed
- The number of candidate entries to be searched is $high - low + 1$
- The number of remaining candidates is either

$$\begin{aligned} & (mid - 1) - low + 1 \\ &= \left\lfloor \frac{low + high}{2} \right\rfloor - low \leq \frac{high - low + 1}{2} \quad \text{or} \quad \begin{aligned} & high - (mid + 1) + 1 \\ &= high - \left\lfloor \frac{low + high}{2} \right\rfloor \leq \frac{high - low + 1}{2} \end{aligned} \end{aligned}$$

- The number of candidates remaining after the i -th call is at most $\frac{n}{2^i}$
- The maximum number of recursive calls m satisfies $\frac{n}{2^m} < 1 \rightarrow m = \lfloor \log n \rfloor + 1$
- Binary search runs in $O(\log n)$ time

Ordered Maps

- Comparing Map Implementations

<i>Method</i>	<i>List</i>	<i>Hash Table</i>	<i>Search Table</i>
size, empty	$O(1)$	$O(1)$	$O(1)$
find	$O(n)$	$O(1)$ exp., $O(n)$ worst-case	$O(\log n)$
insert	$O(1)$	$O(1)$	$O(n)$
erase	$O(n)$	$O(1)$ exp., $O(n)$ worst-case	$O(n)$

Dictionaries

Dictionaries

■ Dictionary

- A data type for storing key-value pairs
- A key-value pair (k, v) is called an entry where k is the key and v is its corresponding value
- Multiple entries with the same key are allowed
- The dictionary ADT supports the following operations:

`size()`: Return the number of entries in D .

`empty()`: Return true if D is empty and false otherwise.

 `find( $k$ )`: If D contains an entry with key equal to k , then return an iterator p referring any such entry, else return the special iterator end.
 arbitrary entry

`findAll( $k$ )`: Return a pair of iterators (b, e) , such that all the entries with key value k lie in the range from b up to, but not including, e .

`insert( $k, v$ )`: Insert an entry with key k and value v into D , returning an iterator referring to the newly created entry.

 arbitrary entry
`erase( $k$ )`: Remove from D an arbitrary entry with key equal to k ; an error condition occurs if D has no such entry.

`erase( $p$ )`: Remove from D the entry referenced by iterator p ; an error condition occurs if p points to the end sentinel.

`begin()`: Return an iterator to the first entry of D .

`end()`: Return an iterator to a position just beyond the end of D .

Dictionaries

Dictionary Example

<i>Operation</i>	<i>Output</i>	<i>Dictionary</i>
empty()	true	$\emptyset$
insert(5,A)	$p_1 : [(5,A)]$	$\{(5,A)\}$
insert(7,B)	$p_2 : [(7,B)]$	$\{(5,A), (7,B)\}$
insert(2,C)	$p_3 : [(2,C)]$	$\{(5,A), (7,B), (2,C)\}$
insert(8,D)	$p_4 : [(8,D)]$	$\{(5,A), (7,B), (2,C), (8,D)\}$
insert(2,E)	$p_5 : [(2,E)]$	$\{(5,A), (7,B), (2,C), (2,E), (8,D)\}$
find(7)	$p_2 : [(7,B)]$	$\{(5,A), (7,B), (2,C), (2,E), (8,D)\}$
find(4)	end	$\{(5,A), (7,B), (2,C), (2,E), (8,D)\}$
find(2)	$p_3 : [(2,C)]$	$\{(5,A), (7,B), (2,C), (2,E), (8,D)\}$
findAll(2)	(p_3, p_4)	$\{(5,A), (7,B), (2,C), (2,E), (8,D)\}$
size()	5	$\{(5,A), (7,B), (2,C), (2,E), (8,D)\}$
erase(5)	—	$\{(7,B), (2,C), (2,E), (8,D)\}$
erase(p_3)	—	$\{(7,B), (2,E), (8,D)\}$
find(2)	$p_5 : [(2,E)]$	$\{(7,B), (2,E), (8,D)\}$

Dictionaries

■ C++ Implementation

- Assume that items with the same keys are stored contiguously

```
class Range {                                // an iterator range
private:
    Iterator _begin;                         // front of range
    Iterator _end;                           // end of range
public:
    Range(const Iterator& b, const Iterator& e) // constructor
        : _begin(b), _end(e) { }
    Iterator& begin() { return _begin; }      // get beginning
    Iterator& end() { return _end; }          // get end
};
```

```
template <typename K, typename V, typename H>
class HashDict : public HashMap<K,V,H> {
public:                                       // public functions
    typedef typename HashMap<K,V,H>::Iterator Iterator;
    typedef typename HashMap<K,V,H>::Entry Entry;
    // ...insert Range class declaration here
public:                                     // public functions
    HashDict(int capacity = 100);          // constructor
    Range findAll(const K& k);              // find all entries with k
    Iterator insert(const K& k, const V& v); // insert pair (k,v)
};
```

Dictionaries

■ C++ Implementation

- Insert the new entry immediately prior to the location found by searching
- In *findAll(k)* operation, *b* is the first entry and *e* is the entry immediately following the last entry with key *k*

```
/* HashDict<K,V,H> :: */           // constructor
HashDict(int capacity) : HashMap<K,V,H>(capacity) { }
```

Code Fragment 9.25: The class HashDict constructor.

```
/* HashDict<K,V,H> :: */           // insert pair (k,v)
Iterator insert(const K& k, const V& v) {
    Iterator p = finder(k);           // find key
    Iterator q = inserter(p, Entry(k, v)); // insert it here
    return q;                         // return its position
}
```

```
/* HashDict<K,V,H> :: */           // find all entries with k
Range findAll(const K& k) {
    Iterator b = finder(k);           // look up k
    Iterator p = b;
    while (!endOfBkt(p) && (*p).key() == (*b).key()) { // find next unequal key
        ++p;
    }
    return Range(b, p);               // return range of positions
}
```

Code Fragment 9.27: An implementation of the dictionary function findAll.